

ENPM691 Homework 03: Variable Placement and Assembly Analysis

Nelay Sharma
University of Maryland
UID: 122295414
9/30/2025

Abstract—Understanding where variables reside in memory is fundamental for system programming and security analysis, both for practical debugging and theoretical analysis. This homework assignment focuses on how the compiler determines the placement of automatic, static, global, and dynamic variables in its memory layout. For example, local variables are typically stored on the stack. In contrast, heap variables are managed in dynamically allocated regions, and global/static variables are stored in fixed sections of the program image (i.e., .data and .bss). Running a test program under GDB (version name) and inspecting the assembly with `objdump` allows us to confirm these expectations and demonstrate the compiler’s code generation patterns.

I. INTRODUCTION

Understanding where variables reside in memory is fundamental for systems programming and security analysis systems for both practical debugging and theoretical analysis. This homework is how the compiler decides to place automatic, static, global, and dynamic variables in its’ memory layout. For example, local variables are expected to be on the stack, while heap variables are managed in dynamically allocated regions, and global/static variables in fixed sections of the program image (i.e., .data, .bss). Running a test program under GDB (version name) and inspecting the assembly with `objdump`, we can confirm these expectations and show the compiler’s code generation patterns. [1] [3]

II. METHODOLOGY

A. Environment

All work was done in a Kali Linux virtual machine inside VMware Workstation Pro 17, with 80 GB of storage and 8 GB of memory. Compilation was performed with GCC 14.2.0 (Debian 14.2.0-1) in 32-bit mode using the `-m32` flag, and debugging/analysis was carried out with GDB 16.3. The architecture used was IA-32 (x86). Multilib support was installed to ensure successful `-m32` compilation. A dedicated directory named `hw3` (inside the Kali VM) was created to hold source code, compiled binaries, and analysis outputs. [2]

B. Programs

The test program `address_layout.c` declares two initialized global, two uninitialized global, two static locals, two automatic locals, and two heap allocations; `address_layout.c` prints its addresses to illustrate the layout. The design of `address_layout.c` captures all the

main storage classes, where we can see their placement side by side in both execution and disassembly. [2]

C. Artifacts

Artifacts collected for this assignment included:

- **Code:** The source program `address_layout.c`, covering local, global, static, and heap variables.
- **Struct:** The implicit memory layout illustrated by variable groupings (stack, heap, data, and BSS).
- **Compiler Flags:** The build used `-m32 -g -fno-pic -no-pic` to force 32-bit output, include debug symbols, and disable position-independent execution.
- **Debugger Steps:** GDB session transcript showing breakpoints, variable inspection, and stack-frame offsets.
- **Disassembly:** Output of `main` from `objdump`, showing frame-relative vs. absolute addressing.
- **Section/Symbol Dumps:** Information from `readelf` and `nm`, mapping variable names to memory regions.
- **Data Collected:** Raw program output with printed addresses, organized into tables for comparison.

III. RESULTS

A. Program Execution

The first run yielded a spread of addresses, the table indicated that the stack locals ended up close together at the top of the memory. At the same time, heap allocations were located in a different range of virtual memory, with globals/statics remaining in the `0x0804c0xx` range. This aligns with the textbook model of a process memory layout, where the stack grows downward, the heap grows upward, and global/static data is stored at fixed offsets within the program image. [3]

TABLE I
OBSERVED VARIABLE ADDRESSES AT RUNTIME

Variable	Storage Class	Address
<code>local_var_1</code>	Automatic (stack)	<code>0xffe0a514</code>
<code>local_var_2</code>	Automatic (stack)	<code>0xffe0a510</code>
<code>ptr_1</code>	Heap (malloc)	<code>0x088e41a0</code>
<code>ptr_2</code>	Heap (malloc)	<code>0x088e4210</code>
<code>global_uninit_var_1</code>	Global (.bss)	<code>0x0804c020</code>
<code>global_uninit_var_2</code>	Global (.bss)	<code>0x0804c024</code>
<code>static_var_1</code>	Static Local (.data)	<code>0x0804c028</code>
<code>static_var_2</code>	Static Local (.data)	<code>0x0804c02c</code>
<code>global_var_1</code>	Global (.data)	<code>0x0804c018</code>
<code>global_var_2</code>	Global (.data)	<code>0x0804c01c</code>

B. Assembly Analysis

The disassembly of the main function showed that local variables were accessed from stack frame offsets, while globals were accessed from fixed absolute addresses. Essentially, the compiler generates different kinds of instructions depending on the variable's lifetime and storage class, where locals rely on relative offsets that change per function call.

At the same time, globals/statics are hard-coded at fixed memory addresses. Table II aligns the variables with how the compiler references them (i.e., `[ebp-0x14]` for a stack local versus `0x804c028` for a static). This shows the compiler uses frame-pointer math (`ebp-...`) for locals, then uses absolute address for globals/statics. [5]

TABLE II
REPRESENTATIVE ASSEMBLY REFERENCES

Variable	Reference Type	Instruction Form
local_var_1	Stack (frame offset)	<code>[ebp-0x14]</code>
local_var_2	Stack (frame offset)	<code>[ebp-0x18]</code>
ptr_1	Stack slot (pointer)	<code>[ebp-0xc]</code>
ptr_2	Stack slot (pointer)	<code>[ebp-0x10]</code>
static_var_1	Absolute address	<code>0x804c028</code>
static_var_2	Absolute address	<code>0x804c02c</code>
global_var_1	Absolute address	<code>0x804c018</code>
global_var_2	Absolute address	<code>0x804c01c</code>

C. Debugger Output

A GDB session at the breakpoint confirmed what we saw in both execution and disassembly. Locals appeared at stack addresses near the frame pointer, malloc pointers started as `0x0` before being updated to real heap addresses, and globals/statics lived at the same fixed addresses seen in the disassembly. [4] [8]

TABLE III
DEBUGGER OBSERVATIONS OF VARIABLES

Variable	Observed Value	Address
local_var_1	0	0xffffcf24
local_var_2	-134482944	0xffffcf20
ptr_1	0x0 (before malloc)	0xffffcf14
ptr_2	0x0 (before malloc)	0xffffcf10
static_var_1	0	0x804c028
static_var_2	0	0x804c02c
global_uninit_var_1	0	0x804c020
global_uninit_var_2	0	0x804c024
global_var_1	0	0x804c018
global_var_2	0	0x804c01c

IV. DISCUSSION AND CONCLUSION

The homework confirms the canonical memory layout in 32-bit Linux processes. Automatic locals are allocated on the stack with frame-pointer offsets, heap allocations grow upwards from the heap segment, and static/global variables live in fixed program sections. The compiler emits relative instructions for locals and absolute references for globals/statics. From a security and debugging perspective, this homework illustrates the importance of understanding this memory layout. For example, buffer overflows typically overwrite stack locals,

while heap spraying or heap overflows exploit the predictable growth of dynamic allocations.

Malware analysts and reverse engineers constantly rely on knowing whether a variable should live in the stack, heap, or data section. These results underscore the importance of understanding memory placement in debugging, reverse engineering, and security.

Even in small programs, a variable's storage class directly impacts its lifetime, scope, and addressing - and once you understand the rules, these behaviors are much less confusing.

ACKNOWLEDGEMENTS

Special thanks to Dr. Lewis Collier and TA Ilya Yatsenko for assistance in answering questions and guidance for this homework assignment.

GLOSSARY OF TERMS

BSS (Block Started by Symbol): A segment of a program's memory image that stores global and static variables that are uninitialized or initialized to zero. The operating system clears this section at program load, so the variables start with zero values.

Heap Allocations: Memory is requested at runtime with functions like `malloc`. It resides in the heap region and grows upward until it is freed.

Frame-Pointer Math: Arithmetic is used to reach variables within a stack frame. For example, `[ebp-0x14]` means 20 bytes (in decimal) below the frame pointer..

Absolute Addresses: A fixed memory address hard-coded into an instruction (i.e., `0x0804c018`, `0x0804c028`, `0x0804c02c`); Common for global and static variables in the `.data` or `.bss` sections.

REFERENCES

- [1] ENPM691 Lecture 03, "Machine Level Programming 1," 2025.
- [2] ENPM691 Course Materials, "programs_export.zip," 2025.
- [3] Linux Foundation, "Special Sections," in *Linux Standard Base Core Specification, Version 1.1*, 2001.
- [4] M. Kerrisk, "elf(5) — Executable and Linkable Format (ELF) file format," *Linux Man Pages*, man7.org, 2024.
- [5] Free Software Foundation, "Frame Layout," in *GCC Internals*, GNU Project, 2024. [Online].
- [6] OSDev Wiki, "ELF," *OSDev.org*, 2024. [Online].
- [7] B. W. Kernighan and D. M. Ritchie, *The C Programming Language*, 2nd ed. Prentice Hall, 1988.
- [8] GCC Online Documentation, "Using the GNU Compiler Collection (GCC)," 2024. [Online]. Available: <https://gcc.gnu.org/onlinedocs/>.

APPENDIX A MAIN SOURCE CODE

```
#include <stdio.h>
#include <malloc.h>
int global_var_1 = 0;
int global_var_2 = 0;
int global_uninit_var_1;
int global_uninit_var_2;

int main() {
    int local_var_1 = 0;
    int local_var_2 = 0;
    static int static_var_1 = 0;
    static int static_var_2 = 0;
    int *ptr_1 = malloc(100);
    int *ptr_2 = malloc(100);
    printf("Local var 1 address: %p\n", &local_var_1);
    printf("Local var 2 address: %p\n", &local_var_2);
    printf("Heap var 1 address: %p\n", ptr_1);
    printf("Heap var 2 address: %p\n", ptr_2);
    printf("Global (uninit) var 1 address: %p\n", &global_uninit_var_1);
    printf("Global (uninit) var 2 address: %p\n", &global_uninit_var_2);
    printf("Static Local var 1 address: %p\n", &static_var_1);
    printf("Static Local var 2 address: %p\n", &static_var_2);
    printf("Global var 1 address: %p\n", &global_var_1);
    printf("Global var 2 address: %p\n", &global_var_2);
    return 0;
}
```

APPENDIX B BUILD/RUN SCRIPTS

```
$ gcc -m32 -g -fno-pic -no-pie address_layout.c -o address_layout
$ ./address_layout | tee run_output.txt
```

APPENDIX C DEBUGGER OUTPUT

```
Breakpoint 1, main () at address_layout.c:12
local_var_1 = 0
local_var_2 = -134482944
static_var_1 = 0
static_var_2 = 0
ptr_1 = 0x0
ptr_2 = 0x0
$1 = (int *) 0xffffcf24
$2 = (int *) 0xffffcf20
$5 = (int *) 0x804c028 <static_var_1>
$6 = (int *) 0x804c02c <static_var_2>
$7 = (int *) 0x804c020 <global_uninit_var_1>
$8 = (int *) 0x804c024 <global_uninit_var_2>
$9 = (int *) 0x804c018 <global_var_1>
$10 = (int *) 0x804c01c <global_var_2>
```

APPENDIX D RAW PROGRAM OUTPUT

```
Local var 1: 0xffe0a514
Local var 2: 0xffe0a510
Heap var 1: 0x088e41a0
Heap var 2: 0x088e4210
Global (uninit) var 1: 0x0804c020
Global (uninit) var 2: 0x0804c024
Static Local var 1: 0x0804c028
Static Local var 2: 0x0804c02c
Global var 1: 0x0804c018
Global var 2: 0x0804c01c
```

APPENDIX E BINARY INSPECTION

```
$ readelf -S address_layout | egrep '\.text|\.data|\.bss'
[13] .text          PROGBITS 08049000 ...
[23] .data           PROGBITS 0804c000 ...
[24] .bss            NOBITS   0804c018 ...
$ nm -n address_layout | egrep 'global_|static_var_'
0804c018 D global_var_1
0804c01c D global_var_2
0804c020 B global_uninit_var_1
0804c024 B global_uninit_var_2
0804c028 D static_var_1
0804c02c D static_var_2
```

APPENDIX F DISASSEMBLY (EXCERPT)

```
08049187: mov    DWORD PTR [ebp-0x14],0x0    ; local_var_1 (stack)
0804918e: mov    DWORD PTR [ebp-0x18],0x0    ; local_var_2 (stack)
0804919a: call   0x08049050 <malloc@plt>    ; allocate
08049206: push   0x0804c020                  ; &global_uninit_var_1 (.bss)
08049230: push   0x0804c028                  ; &static_var_1 (.data)
```

APPENDIX G /PROC MAPS (EXCERPT)

```
555858559000-555858751000 rw-p 00000000 00:00 0    [heap]
7fffd18149000-7fffd1816f000 rw-p 00000000 00:00 0    [stack]
... libc.so.6 mapped (r-xp / r--p / rw-p segments) ...
... ld-linux-x86-64.so.2 loader mappings ...
```

APPENDIX H COMPILER AND DEBUGGER FLAGS

- -m32: Compile for 32-bit IA-32 architecture.
- -g: Include debugging information for GDB.
- -fno-pic: Disable position-independent code generation.
- -no-pie: Disable PIE (Position Independent Executable) support.